Smart Dukan v2 — AI Build Specification

Oct 13, 2026 · @Ali AHAMD

0. Instructions for the AI builder

You are building **Smart Dukan**, a production-quality, offline-first Android app for Pakistani shopkeepers, starting with mobile phone shops. This document is the single source of truth. Read it fully before writing code.

How to work

1. Build in the milestone order in Section 11. Finish, test and show each milestone before starting the next.
2. Never skip the accounting rules in Section 8. Money and stock must always reconcile to the paisa.
3. Write clean, modular, documented Flutter code. Every feature lives in its own folder (Section 6).
4. Write unit tests for every posting rule and every repository method that writes money or stock.
5. All user-facing text comes from localization files. Never hardcode strings in widgets.
6. If something in this spec is unclear, choose the simplest safe option, write it down in `DECISIONS.md`, and continue.
7. Do not add cloud, login, analytics SDKs or ads in V1. The app must work in airplane mode from first launch.
8. After each milestone, output: what was built, files changed, how to run it, tests passing, known gaps.

Stop rule: Do not build the whole app in one go. Work one milestone at a time, and the owner tests each one on a real Android phone. If any accounting or stock test fails, stop and fix it before starting the next milestone.

Quality bar

- Looks like a modern, premium fintech app: clean, calm, generous spacing, large touch targets.
- A shopkeeper with basic literacy can record a sale in under 10 seconds.
- Runs smoothly on a 3 GB RAM Android phone (Android 8.0+, API 26).
- No crash, no data loss, no wrong balance. Correctness beats features.

1. Product summary and positioning

Smart Dukan is a private, no-login business app that keeps a shop's sales, stock, udhaar and cash on the phone itself, with deep features for mobile phone shops.

Tagline (Roman Urdu): "Aapka hisaab, sirf aapke phone par."

Primary user: owner of a small mobile phone and accessories shop in Pakistan (1–5 staff, 200–3,000 items, sells new and used phones, does repairs, sells on qist).

Secondary users (same core, optional packs later): kiryana, garments, pharmacy, wholesale.

Why this exists. Free apps (Sauda, Udhaar Book, Easy Khata, DigiKhata, Asaan Hisab) already offer generic khata, stock and invoices. Smart Dukan wins on four points:

Differentiator	What it means in the product
No account, no server	No sign-up, no phone OTP. All data in an encrypted local database. Export anytime.
Mobile-shop depth	IMEI tracking, used-phone buy/sell with seller record, repair jobs, qist plans.
Done-for-you setup	Excel/CSV import plus a paid setup service so the shop is live on day one.
Works on cheap phones	Fast cold start, small APK, no internet ever required.

Product principles

- Speed at the counter first; reports second.
- Shopkeeper words, not accountant words: Lena, Dena, Udhaar, Wasooli, Galla, Qist.
- Every number on screen can be tapped to see the entries behind it.
- Mistakes are corrected by reversal, never by silent deletion.

2. Business model, pricing and licensing

The core app is free and unlimited; money comes from one-time Pro packs and a paid setup service. Prices below are launch hypotheses to test in the pilot.

Tier	Price (PKR)	Unlocks
Free Core	0	Sales, purchases, products, stock, customers, suppliers, udhaar, expenses, cash closing, basic reports, backup/restore
Mobile Shop Pro (one-time, per device)	1,999 (launch 1,499)	IMEI tracking, used-phone buy/sell, repair jobs, qist plans, branded PDF invoices, staff roles, advanced reports
Setup service (outside the app)	2,000–5,000 per shop	Stock entry, opening balances, training
Future packs	TBD	Pharmacy (batch/expiry), Wholesale (cartons, routes), Garments (size/colour)

Rules

- Never lock a user out of their own data. If Pro lapses or is not bought, Pro records stay readable and exportable.
- The free tier must never show ads.

License implementation (offline, no server)

1. On first launch, generate a random 128-bit install ID (UUID v4) and store it in Android Keystore-backed secure storage and inside the encrypted database.
2. Show the user the **Device Code** = the full install ID in Crockford base32 (26 characters), grouped in fives (`K7QM2-XPA9D-...`), with a Copy button and a "Send on WhatsApp" button. Never truncate it.
3. User pays via JazzCash/Easypaisa/bank and sends the Device Code on WhatsApp.
4. The seller runs `tools/license_gen.dart` (Milestone 9), which signs `deviceCode|pack|issuedDate|licenseId` with an **Ed25519 private key**. The private key lives only on the seller's laptop (plus an offline backup). It is never in the Flutter app, the Git repository or any cloud folder; add the key path to `.gitignore`.
5. The user receives the code as a link `smartdukan://activate?c=<base64url>` or a QR. Tapping it opens the app.
6. The app verifies the signature with the embedded **public key**, checks the device code matches, and unlocks the pack. The licence is stored in secure storage and in the database.
7. Any invalid, altered or wrong-device code is rejected with a clear message. Business records are never touched by licence checks.

Keep a Google Play in-app purchase path behind a feature flag (off at launch), since many buyers have no card and Play policy must be verified first.

Licence lifecycle

Situation	What happens
App updated	Install ID and licence kept; nothing to do
App reinstalled or phone reset	New install ID; data comes back only from a backup. Restored data stays readable; Pro unlocks again after a free re-activation (seller checks the old licenseId)
Secure storage lost but DB intact	App reads the install ID from the encrypted DB; if they disagree, show Re-activate screen; data stays readable
New phone	Restore backup, then a Transfer request with old and new Device Codes; seller issues a new code and marks the old licenseId as transferred in a simple sheet

Known limitation (state it in the licence terms): an offline app cannot revoke a copied or transferred licence. Device binding makes casual sharing hard but cannot stop determined piracy. Keep a simple sales register (licenseId, device code, buyer phone, date) to handle transfers and disputes.

Launch payment decision: start with manual JazzCash/Easypaisa activation codes. Add Google Play billing only after checking current Play payment policy for this app's distribution and pricing.

3. Scope by version

V1 ships the free core plus the Mobile Shop Pro pack; everything else waits for pilot feedback.

V1 is built and released in four phases. Do not start a phase until the previous one passes its tests on a real phone.

Phase	Contents	Milestones (Section 11)	Exit gate
A — Accounting foundation	Database, products, stock movements, sales, purchases, payments, balances, automated tests	1, 2, 4, 5 (sale core)	Section 8 worked example and all invariants pass
B — Pilot-ready app	Dashboard, khata, expenses, Galla Band, basic reports, encrypted backup/restore, English + Roman Urdu	3, 5 (polish), 6, 7	14-day pilot in 3 shops matches the paper register
C — Mobile Shop Pro	IMEI, used phones, repairs, qist, Urdu script (RTL), paid activation	8, 9	Pro features tested in pilot shops; licence tests pass
D — Public release	Security review, performance tests, Play Store requirements, pilot fixes	10	Full Section 10 checklist green

The feature list below is the full V1 target across all four phases, not a single build.

V1 — Launch (build now)

- Onboarding: language, shop profile, business type, optional demo data, PIN setup
- Dashboard ("Aaj ka Hisaab")
- Products, categories, brands, units, variants, opening stock, barcode/camera scan
- Sales (POS): cash, udhaar, mixed, discount, returns, printable/shareable receipt
- Purchases from suppliers, purchase returns
- Parties: customers and suppliers with running khata
- Payments in/out, allocation to invoices
- Expenses with categories
- Cash closing ("Galla Band"): expected vs counted cash
- Reports: daily summary, sales, profit, stock value, udhaar list, party ledger
- Mobile Shop Pro: IMEI per unit, used-phone purchase with seller details and photos, repair jobs, qist plans and schedules
- WhatsApp sharing of receipts and payment reminders (via share intent, no API)
- Encrypted local backup and restore; CSV/Excel import and export
- PIN/biometric lock, staff roles, audit log

- Offline license activation

V1.1 — After pilot

- Bluetooth 58mm/80mm thermal printer
- Low-stock and qist due-date local notifications
- Daily auto-backup to a user-chosen folder or Google Drive (user's own account)
- Price list share as image/PDF

V2 — Only with proven demand

- Optional paid cloud backup and multi-device sync
- Pharmacy, Wholesale and Garments packs
- FBR/PRA invoice integration through a licensed integrator
- Easyload and bill-payment commissions through a partner

Out of scope for V1: cloud accounts, online store, payroll, AI features, iOS, desktop, ads, analytics SDKs.

4. Design system

The look is calm, trustworthy and premium: deep green brand, white cards, big numbers, Material 3 components with custom theming. Support light and dark mode.

Colour tokens (define in `core/theme/app_colors.dart`, never use raw hex in widgets)

Token	Light	Dark	Use
primary	#0F6E56	#4FC3A1	Brand, main buttons, active tab
primaryContainer	#D7F2E8	#12352C	Selected chips, highlights
secondary	#F2A541	#F5B867	Accent: Pro badge, attention
success (Lena / money in)	#1E8E3E	#5BD17A	Received, profit
danger (Dena / money out)	#C62828	#EF6B6B	Payable, loss, delete
warning	#E37400	#FFB74D	Low stock, due soon
background	#F6F8F7	#0F1412	Screen background
surface	#FFFFFF	#1A211E	Cards, sheets
textPrimary	#13201B	#E8EFEC	Main text
textSecondary	#5B6B65	#9DB0A8	Labels, hints

Token	Light	Dark	Use
outline	#DCE3E0	#2C3733	Borders, dividers

Typography

- English and Roman Urdu: **Inter** (bundled, not downloaded). Numbers use tabular figures.
- Urdu: **Noto Nastaliq Urdu** for headings and **Noto Naskh Arabic** for body and dense lists (Nastaliq is too tall for tables).
- Scale: Display 32/40 bold (big money figures), Title 20/28 semibold, Body 16/24, Label 14/20 medium, Caption 12/16.
- Money format: `Rs 125,000` (groups of three, no decimals unless paisa exist). Do not use the lakh grouping `1,25,000`.

Layout and components

- 8-point spacing grid; screen padding 16; card radius 16; button radius 12; minimum touch target 48×48.
- Primary buttons full-width, height 56, bold label, icon on the leading side.
- Bottom navigation with 5 tabs: Home, Sale, Khata, Stock, More. A large centre "+ Nayi Sale" floating action.
- Amount fields use a custom numeric keypad sheet with big keys and a "000" key.
- Lena shown in green with a down-arrow icon; Dena in red with an up-arrow icon. Never rely on colour alone.
- Empty states: friendly illustration (simple vector), one line of help, one clear button.
- Every destructive action uses a confirmation bottom sheet; financial corrections create a reversal.
- Snackbar with "Undo" for 5 seconds after saving a sale (Undo creates a reversal, not a delete).

Language and RTL

- Three display modes: English, Roman Urdu, Urdu (RTL). Switchable from Settings and from onboarding.
- In Urdu mode the whole layout mirrors (`Directionality.rtl`), except numbers, IMEI, phone numbers and SKUs, which stay LTR.
- Use Western digits (0–9) in all modes by default; optional Urdu digits setting.

Motion and feel

- 200ms ease-out transitions; subtle haptic on sale saved and payment received.
- Success screen after a sale: big tick, amount, and three actions: Share, Print, New Sale.

5. Screens and user flows

Every screen below must exist in V1. Labels are given in Roman Urdu / English; Urdu script lives in the localization file.

5.1 Onboarding (first launch, under 60 seconds)

1. Splash with logo, then language picker: English, Roman Urdu, اردو.
2. Shop setup: shop name, owner name, phone (optional), city, business type (Mobile Shop, Kiryana, Garments, Pharmacy, Wholesale, Other).
3. "Demo data ke saath dekhein?" Yes loads a sample shop that can be wiped with one button; No starts clean.
4. Set a 4-digit PIN and optionally enable fingerprint.
5. Land on Home with a 3-step coach mark: New Sale, Add Stock, Khata.

5.2 Home — "Aaj ka Hisaab"

- Header: shop name, date, sync-free badge "Offline • Mehfooz".
- Four stat cards: Aaj ki Sale, Galla (cash in hand), Lena (customer dues), Dena (supplier dues).
- Second row: Aaj ka Munafa (estimated gross profit), Low stock count, Qist due today (Pro), Repairs ready (Pro).
- Quick actions grid: Nayi Sale, Purchase, Payment Lein, Payment Dein, Kharcha, Galla Band.
- Recent activity list (last 10 entries), each tappable.

5.3 New Sale (POS) — the most important screen

1. Search bar with auto-focus; search by name, SKU, barcode, or IMEI. Camera scan button.
2. Grid/list toggle of products with price and stock. Tap adds 1; long-press opens quantity and price edit.
3. For IMEI-tracked items, adding opens a picker of available IMEIs (scan or select).
4. Cart sheet at bottom: items, quantity steppers, line discount, total.
5. Customer: default "Walk-in"; tap to pick or quick-add (name + phone).
6. Payment: Cash / Udhaar / Mixed / Bank-Wallet. Mixed shows two amount fields. Udhaar requires a named customer.
7. Save is disabled while saving (prevents double invoices). One DB transaction posts everything.
8. Success screen: Share on WhatsApp (PDF/image), Print, New Sale.

Target: a known item, cash sale, in 3 taps.

5.4 Purchase

Same layout as Sale but for suppliers: supplier, items with cost price, paid amount, due amount, bill photo (optional). Updates stock and weighted average cost.

5.5 Stock (products)

- List with filters: category, brand, low stock, out of stock, IMEI items.
- Product form: name, category, brand, SKU (auto), barcode(s), unit, sale price, cost price, min stock alert, tracks IMEI (yes/no), has variants (colour/storage), opening stock and opening cost.
- Product detail: current stock, stock value, movement history, IMEI list with status (In stock / Sold / Returned / Under repair).
- Stock adjustment: damaged, lost, found, count correction, each with a reason.
- Import from Excel/CSV with column mapping preview and error rows report.

5.6 Khata (parties)

- Tabs: Customers, Suppliers. Sorted by balance; search.
- Party card: name, phone, balance with Lena/Dena colour, last activity.
- Party ledger: date, description, debit, credit, running balance; filter by date.
- Actions: Payment received/given, Send reminder on WhatsApp (pre-filled Roman Urdu message with balance), Call, Share statement PDF.
- Opening balance on creation; credit limit (warns at sale time).

5.7 Payments and Expenses

- Payment In / Out: party, amount, method (cash, bank, JazzCash, Easypaisa), note, allocate to open invoices (auto oldest-first, editable).
- Expense: category (rent, bijli, salary, chai-pani, transport, other), amount, method, note, receipt photo.

5.8 Galla Band (cash closing)

Shows opening cash, cash sales, cash received, cash paid, expenses, expected closing. User enters counted cash (with note denomination helper: 5000, 1000, 500, 100, 50, 20, 10). Difference saved as "Cash short/over" entry. Closing locks the day's cash entries from casual editing (corrections still allowed via reversal with owner PIN).

5.9 Mobile Shop Pro screens

- **IMEI search:** type or scan an IMEI to see full history: purchased from, cost, sold to, date, warranty, repairs.
- **Used phone purchase:** seller name, CNIC number, phone, CNIC front/back photos, device photos, model, IMEI(s), condition grade (A/B/C), price paid, payment method. Prints a simple purchase receipt for the seller to sign.
- **Repair jobs:** create job with customer, device, IMEI, problem, estimate, advance, expected date, technician. Status: Received → Diagnosing → Waiting parts → Ready → Delivered / Returned unrepaired. Parts used deduct stock. Token slip print/share. Ready notification message on WhatsApp.
- **Qist (installment) plans:** product sold, total price, down payment, number of instalments, frequency (weekly/monthly), guarantor name and phone, start date. Auto-generates schedule. Collect instalment screen. Overdue list on Home.

5.10 Reports

Date-range picker (Today, Yesterday, This week, This month, Custom). Reports: Daily summary, Sales by item/category/staff, Profit & Loss (simple), Stock value and stock list, Low stock, Customer dues ageing, Supplier dues, Expenses by category, Cash book, IMEI sold list (Pro), Repairs (Pro), Qist collection and overdue (Pro). Each exports to PDF and CSV.

5.11 More / Settings

Shop profile and logo, invoice settings (prefix, footer, terms), language, theme, PIN and biometric, staff and roles, backup and restore, import/export, license and Pro packs, help videos (links), about, privacy policy.

6. Tech stack and architecture

Flutter with a feature-first clean architecture, Riverpod for state, and Drift on an encrypted SQLite database. Use the latest stable versions at build time and pin them in `pubspec.lock`.

Concern	Choice	Notes
Framework	Flutter (stable), Dart 3	Android only; minSdk 26, targetSdk = current Play requirement
State management	flutter_riverpod + riverpod_generator	One provider per use case

Concern	Choice	Notes
Navigation	go_router	Typed routes, deep link `smartdukan://activate`
Database	drift + sqlcipher_flutter_libs	Encrypted SQLite, typed queries, migrations
Secure storage	flutter_secure_storage	DB key, install ID, license
Models	freezed + json_serializable	Immutable entities
Localization	flutter_localizations + intl (ARB files)	en, ur (Urdu script), ur_Latn style keys via a third ARB
Barcode/IMEI scan	mobile_scanner	Camera scanning
PDF and print	pdf + printing	Invoices, statements, reports
Sharing	share_plus, url_launcher	WhatsApp share intent, calls
Files	file_picker, path_provider	Backup, CSV/Excel import
Excel	excel, csv	Import/export
Auth lock	local_auth	Fingerprint/face
Crypto	cryptography	Ed25519 verify, AES-GCM backups
Images	image_picker + flutter_image_compress	CNIC, bills, device photos stored in app-private storage
Tests	flutter_test, mocktail, drift in-memory DB	Unit, widget, integration

Layers (strict direction: presentation → application → domain ← data)

- `presentation`: widgets, screens, Riverpod notifiers. No SQL here.
- `application`: use cases such as `CreateSale`, `ReceivePayment`, `CloseCashDay`. Each runs in ONE database transaction.
- `domain`: entities, value objects (`Money` as integer paisa), posting rules, validation.
- `data`: Drift tables, DAOs, repositories, import/export, backup.

Folder structure

```
lib/
  main.dart
  app/ (router.dart, app.dart, bootstrap.dart)
  core/
```

```
    theme/ (app_colors.dart, app_typography.dart, app_theme.dart)
    l10n/ (app_en.arb, app_ur.arb, app_ru.arb)  // ru = Roman Urdu keys
    money/ (money.dart, money_format.dart)
    db/ (app_database.dart, tables/, migrations/)
    security/ (pin_service.dart, license_service.dart, crypto.dart)
    widgets/ (amount_keypad.dart, stat_card.dart, party_tile.dart,
empty_state.dart, ...)
    utils/
  features/
    onboarding/   dashboard/   products/   sales/   purchases/
    parties/      payments/    expenses/   cash_closing/
    reports/      settings/    backup/     license/
    pro_mobile/ (imei/, used_phones/, repairs/, qist/)
      each feature: data/, domain/, application/, presentation/
test/  (mirrors lib/)
tools/license_gen.dart
DECISIONS.md
```

Engineering rules

- Money is `int` paisa everywhere. Never use `double` for money.
- All IDs are UUID v7 strings (sortable, safe for a future sync).
- Every table has `created_at`, `updated_at`, `created_by` and `business_id`.
- Posted documents are immutable; corrections create a reversal document linked by `reverses_id`.
- Use `drift` schema versions with tested step-by-step migrations; never drop user data.
- Heavy reports run in an isolate; lists use pagination (50 rows).

7. Database schema

All tables also carry `id` (UUID v7, PK), `business_id`, `created_at`, `updated_at`, `created_by`, and `is_void` where noted. Money columns are INTEGER paisa; quantities are INTEGER in the smallest unit (e.g. pieces, grams).

Core

Table	Key columns
businesses	name, owner_name, phone, city, address, business_type, logo_path, invoice_prefix, invoice_footer, currency='PKR'

Table	Key columns
users	name, role (owner/manager/cashier/technician), pin_hash (Argon2id or PBKDF2), is_active
app_settings	key, value (language, theme, digits, auto_backup, etc.)
audit_logs	entity, entity_id, action (create/reverse/update_master/login/export), before_json, after_json, user_id
licenses	pack, device_code, issued_at, signature, status
schema_migrations	version, applied_at

Catalogue and stock

Table	Key columns
categories	name, parent_id
brands	name
units	name, symbol, base_unit_id, factor (e.g. carton = 24 pcs)
products	name, name_ur, sku, category_id, brand_id, unit_id, sale_price, min_sale_price, avg_cost, min_stock, tracks_imei, has_variants, is_active
product_variants	product_id, label (e.g. "128GB Black"), sku, sale_price, avg_cost
barcodes	product_id, variant_id, code (unique)
stock_movements	product_id, variant_id, qty_change (+/-), unit_cost, type (opening/purchase/sale/sale_return/purchase_return/adjustment/repair_part/used_purchase), ref_type, ref_id, reason
stock_balances	product_id, variant_id, qty_on_hand, value (kept in sync inside the same transaction; rebuildable from movements)

Table	Key columns
imei_units	product_id, variant_id, imei1, imei2, serial, status (in_stock/sold/returned/in_repair/written_off), purchase_ref_id, sale_ref_id, cost, condition_grade, warranty_until

Parties and documents

Table	Key columns
parties	type (customer/supplier/both), name, phone, cnic, address, opening_balance, credit_limit, notes
sales	invoice_no (unique per business), party_id, date, subtotal, discount, tax, delivery, total, paid, due, payment_mode, status (posted/reversed), reverses_id, note
sale_items	sale_id, product_id, variant_id, imei_unit_id, qty, unit_price, discount, line_total, unit_cost_snapshot
purchases	bill_no, party_id, date, subtotal, discount, total, paid, due, status, reverses_id, bill_photo_path
purchase_items	purchase_id, product_id, variant_id, qty, unit_cost, line_total
payments	direction (in/out), party_id, date, amount, method (cash/bank/jazzcash/easypaisa/other), account_id, reference, status, reverses_id
payment_allocations	payment_id, doc_type (sale/purchase/qist), doc_id, amount
expenses	category_id, date, amount, method, account_id, note, photo_path, status, reverses_id
expense_categories	name

Accounting

Table	Key columns
accounts	code, name, type (asset/liability/equity/income/expense), system_key (CASH, BANK, WALLET, AR, AP, INVENTORY, SALES, COGS, DISCOUNT, EXPENSE_*, CASH_OVER_SHORT, OWNER_CAPITAL)
journal_entries	date, source_type, source_id, memo
journal_lines	entry_id, account_id, party_id (nullable), debit, credit
cash_closings	date, opening_cash, expected_cash, counted_cash, difference, denominations_json, closed_by

Mobile Shop Pro

Table	Key columns
used_phone_purchases	seller_party_id, seller_cnic, cnic_front_path, cnic_back_path, device_photos_json, model, imei_unit_id, grade, price, method, purchase_id
repair_jobs	job_no, party_id, device_model, imei, problem, estimate, advance, final_amount, status, technician_id, expected_date, delivered_at, sale_id
repair_parts	job_id, product_id, qty, unit_cost, charge
repair_status_log	job_id, from_status, to_status, note
qist_plans	sale_id, party_id, guarantor_name, guarantor_phone, total, down_payment, installments, frequency, start_date, status (active/completed/defaulted)
qist_schedule	plan_id, due_date, amount_due, amount_paid, status (due/partial/paid/overdue)

Indexes: products(name), barcodes(code), imei_units(imei1), imei_units(imei2), parties(name, phone), sales(date), sales(invoice_no), journal_lines(account_id), journal_lines(party_id), qist_schedule(due_date, status).

8. Accounting and stock posting rules

Every business event writes its document, its stock movements and a balanced journal entry (total debit = total credit) in one database transaction; if any part fails, nothing is saved. Cost method: weighted average cost per product/variant; IMEI items use the unit's own cost.

Event	Debit	Credit	Stock
Opening stock	Inventory	Owner Capital	+ qty at opening cost
Party opening balance (customer owes)	AR (party)	Owner Capital	—
Purchase (paid part / due part)	Inventory (total)	Cash/Bank (paid) + AP party (due)	+ qty, recompute avg cost
Sale	Cash/Bank (paid) + AR party (due) + Discount	Sales (gross)	− qty
Cost of the sale	COGS	Inventory	(same transaction as sale)
Payment received	Cash/Bank/Wallet	AR party	—
Payment made	AP party	Cash/Bank/Wallet	—
Expense	Expense:category	Cash/Bank	—
Sale return	Sales Returns	Cash or AR party	+ qty; reverse COGS at original cost snapshot
Stock adjustment (loss)	Stock Loss expense	Inventory	− qty
Cash closing difference	Cash Over/Short	Cash (short) or reverse (over)	—
Repair job delivered	Cash/AR	Repair Income; parts: COGS / Inventory	− parts
Qist sale	AR party (full due after down payment) + Cash (down payment)	Sales	− qty; schedule rows created

Event	Debit	Credit	Stock
Reversal of any document	Mirror of the original lines	Mirror of the original lines	Mirror of original movements

8.1 Detailed posting rules

These rules override the summary table above wherever they are more specific. Implement and test every one of them in Milestone 2, before any screen.

Extra system accounts: OPENING_EQUITY, SALES_RETURNS, SALES_TAX_PAYABLE, DELIVERY_INCOME, REPAIR_INCOME, DEFERRED_MARKUP, MARKUP_INCOME, BAD_DEBT, STOCK_LOSS, INVENTORY_ADJUSTMENT, CASH_OVER_SHORT.

Opening balances (go-live date)

Item	Debit	Credit
Opening cash / bank / wallet	Cash / Bank / Wallet	OPENING_EQUITY
Opening stock	Inventory (qty × opening cost)	OPENING_EQUITY
Customer owes the shop	AR (party)	OPENING_EQUITY
Shop owes a supplier	OPENING_EQUITY	AP (party)

All opening entries are dated the go-live date. Corrections are made by reversal plus a new entry.

Invoice extras

- Invoice discount: Dr DISCOUNT (contra-income). For returns, the discount is prorated across lines by their net amount.
- Sales tax charged: Cr SALES_TAX_PAYABLE; Sales is recorded excluding tax. Tax is off by default.
- Delivery charged to the customer: Cr DELIVERY_INCOME.
- Purchase tax and freight paid to the supplier are added to inventory cost in V1 (no input-tax claim). Confirm with an accountant before adding tax features.

Customer advance (paid before any invoice): Dr Cash, Cr AR (party). The party shows a credit balance ("Advance"), which the next sale uses up automatically.

Sale return, including mixed-payment refunds. Let R = returned value (after prorated discount and tax).

1. First reduce what the customer still owes on that invoice: Dr SALES_RETURNS, Cr AR (party), up to the invoice's remaining due.
2. For any remainder, the user chooses: refund now (Dr SALES_RETURNS, Cr Cash/Bank/Wallet) or keep as customer credit (Dr SALES_RETURNS, Cr AR, leaving a credit balance).
3. Stock: + qty; Dr Inventory, Cr COGS at the line's `unit_cost_snapshot`.
4. Returned quantity can never exceed sold quantity minus earlier returns.

Purchase return. Let R = returned qty × original purchase line unit cost.

1. First reduce what the shop still owes the supplier: Dr AP (party), Cr Inventory, up to the bill's remaining due.
2. Remainder: supplier refunds now (Dr Cash/Bank, Cr Inventory) or keeps it as shop credit (Dr AP, leaving a debit balance).
3. Stock: − qty. Recompute average cost = (stock value − R) / (qty − returned qty). If quantity reaches zero and value is not zero, post the leftover to INVENTORY_ADJUSTMENT.
4. Cannot return more than purchased minus earlier returns, nor more than is on hand.

Repair jobs

Event	Debit	Credit
Job created	—	— (no entry)
Advance received	Cash/Wallet	AR (party) (credit balance)
Part issued to job	COGS (repair parts) at avg cost	Inventory
Unused part returned to stock	Inventory	COGS (repair parts)
Job delivered (final bill = labour + parts charge)	AR (party)	REPAIR_INCOME
Payment at delivery	Cash/Wallet	AR (party)
Returned unrepaired with inspection fee	AR (party)	REPAIR_INCOME (fee only)
Advance refunded	AR (party)	Cash/Wallet

Outstanding repair balance = the party's AR lines linked to the job. Parts issued and not returned stay as cost even if the job is returned unrepaired.

Qist (instalment) plans. Let P = cash price of goods, M = markup, T = P + M, D = down payment.

Event	Debit	Credit
Qist sale	AR (party) T	Sales P; DEFERRED_MARKUP M
Cost of goods	COGS	Inventory
Down payment	Cash	AR (party) D
Instalment received (amount A)	Cash	AR (party) A
Markup earned on that instalment	DEFERRED_MARKUP (A × M / T)	MARKUP_INCOME
Cancellation, device taken back	Inventory (agreed recovery value) + DEFERRED_MARKUP (unearned remainder)	AR (party)
Remaining amount written off (owner PIN)	BAD_DEBT	AR (party)

- The down payment also earns markup by the same A × M / T rule. A plan with M = 0 skips the markup lines.
- Instalments are applied to schedule rows oldest first. A row is `partial` when 0 < paid < due.
- `overdue` is a status only, with no journal entry: due_date earlier than today and paid less than due.
- Plan status: active, completed (AR on plan = 0), cancelled.

Cash closing (Galla Band). Difference D = counted cash − expected cash.

- D < 0 (short): Dr CASH_OVER_SHORT, Cr Cash, amount |D|.
- D > 0 (excess): Dr Cash, Cr CASH_OVER_SHORT, amount D.
- D = 0: no entry. The closing record is saved either way.

Worked example (must pass as an automated test)

1. Opening: 10 chargers at Rs 300 cost. Inventory = Rs 3,000.
2. Purchase 10 more at Rs 360, pay Rs 2,000 cash, Rs 1,600 due. New avg cost = (3,000 + 3,600) / 20 = Rs 330. Supplier balance = Rs 1,600 Dena.
3. Sell 4 chargers at Rs 500 to customer Ali: total Rs 2,000, Ali pays Rs 1,200 cash, Rs 800 udhaar. COGS = 4 × 330 = Rs 1,320. Gross profit = Rs 680. Stock = 16.

4. Ali pays Rs 300 later. Ali balance = Rs 500 Lena. No new sale is created.
5. Ali returns 1 charger. Sales return Rs 500 reduces Ali's balance to Rs 0; stock = 17; COGS reversed by Rs 330.
6. Trial balance must equal zero; stock_balances must equal the sum of stock_movements; party balances must equal the sum of their journal lines.

Invariants (check in tests and in a hidden "Health Check" screen)

- Sum of all debits = sum of all credits.
- For every product, qty_on_hand = Σ stock_movements.qty_change, and never negative unless the "allow negative stock" setting is on.
- An IMEI can be `in_stock` in only one place and sold only once until returned.
- Invoice numbers are sequential per business with no gaps; reversed invoices keep their number.
- Party balance shown on screen = Σ (debit − credit) of its AR/AP lines.

9. Security, backup, localization and performance

Data never leaves the phone unless the user exports or shares it. Full backups are always encrypted; everyday invoices and reports are normal readable files.

Security

- Database encrypted with SQLCipher; a random 256-bit key generated on first launch and kept in Android Keystore via flutter_secure_storage.
- App lock: 4–6 digit PIN (hashed with a slow KDF and salt) plus optional biometrics; lock after 2 minutes in background (configurable).
- Roles: Owner (everything), Manager (no settings, no license, no delete of masters), Cashier (sales, payments in, view stock; no cost prices, no profit, no reports), Technician (repair jobs only).
- Sensitive actions require owner PIN: reversals, price below min_sale_price, stock adjustment, restore, export.
- All photos (CNIC, bills, devices) are encrypted at rest with AES-256-GCM using a key held in Android Keystore, stored in app-private storage, and never written to the gallery or cache in plain form.
- `android:allowBackup="false"`; FLAG_SECURE on screens showing CNIC; no logging of personal data.

Personal data (CNIC and seller records)

- Collect the minimum: CNIC number and photos are optional fields, off by default, enabled per shop in Settings.

- Only Owner and Manager roles can view CNIC fields and photos; every view is written to `audit_logs`.
- CNIC data never appears in logs, crash output, PDF/CSV reports or WhatsApp shares.
- Retention setting (default 12 months after the sale of that device). Expired CNIC photos are deleted and the file is overwritten; the purchase record stays without them.
- Owner can delete a seller's identity data on request; the financial record remains.
- If this cannot be built and tested safely in Phase C, ship used-phone purchase without CNIC photos and add them later.

Exports and sharing (three separate kinds)

Kind	Format	Protection
Full backup	`.sdbk` (DB + photos)	Always encrypted with a backup password; owner PIN required
Reports, statements, invoices	PDF / CSV	Readable files; owner PIN for full reports; one-time warning that the file contains business data
WhatsApp receipt or reminder	PDF/image or text	Contains only that customer's own invoice or balance

Backup and restore

- Backup file `SmartDukan_<shop>_<yyyyMMdd_HHmm>.sdbk` = zip of DB + photos, encrypted with AES-256-GCM using a key derived from a user backup password (show a strong warning: lost password = lost backup).
- Destinations via system file picker: phone storage, USB, Google Drive app, WhatsApp-to-self.
- Before any restore: automatically back up current data first, then verify file integrity and schema version, then migrate if older.
- Reminder banner if no backup in 7 days.

Localization

- Three ARB files: `app_en.arb`, `app_ur.arb` (Urdu script), `app_ru.arb` (Roman Urdu, custom locale code mapped in code).
- Glossary to use consistently: Sale = Sale/بیچ, Purchase = Khareedari, Customer = Gahak, Supplier = Supplier, Receivable = Lena, Payable = Dena, Credit = Udhaar, Collection = Wasooli, Cash drawer = Galla, Expense = Kharcha, Profit = Munafa, Instalment = Qist, Stock = Stock/Maal.

- WhatsApp reminder template (Roman Urdu): "Assalam o Alaikum {name}, {shop} ki taraf se yaad dehani: aapka baqaya Rs {amount} hai. Shukriya."

Performance targets

- Cold start under 2 seconds on a 3 GB RAM phone; search results under 150 ms with 5,000 products.
- Sale save under 300 ms; reports for one year of data under 3 seconds.
- Release APK/AAB under 30 MB; use deferred loading for PDF and Excel packages if needed.

Legal and store

- Privacy policy page (data stays on device, what photos are stored and why).
- Terms for the one-time license: per device, includes bug fixes and compatibility updates, transfer process.
- Play Data Safety form: no data collected or shared.

10. Testing and acceptance criteria

A milestone is done only when its checklist passes, all tests are green, and the Health Check screen shows zero errors on the demo data.

Automated tests (minimum)

- Unit: `Money` arithmetic and formatting; every posting rule in Section 8; average cost; invoice numbering; qist schedule generation; license signature verify (valid, tampered, wrong device).
- Repository/integration (in-memory Drift): worked example in Section 8 end to end; reversal of each document type; migration from every previous schema version.
- Widget: New Sale flow, Udhaar requires customer, double-tap Save creates one invoice, RTL layout renders without overflow.
- Golden tests for Home and Sale screens in English and Urdu, light and dark.

Manual acceptance checklist

- [ ] Full sale, purchase, payment and expense in airplane mode from a fresh install
- [ ] Cash, udhaar, mixed and wallet payments all post correctly
- [ ] Partial payment and oldest-first allocation
- [ ] Sale return and purchase return restore stock and balances
- [ ] Carton to piece conversion is correct
- [ ] IMEI cannot be sold twice; IMEI search shows full history
- [ ] Used phone purchase saves CNIC photos privately

- [ ] Repair job moves through all statuses; parts deduct stock
- [ ] Qist plan generates schedule; partial instalment and overdue list work
- [ ] Galla Band difference posts to Cash Over/Short
- [ ] Cashier role cannot see cost or profit
- [ ] Kill the app mid-save: no half-saved documents after restart
- [ ] Backup, wipe app, restore: every balance identical
- [ ] 5,000 products and 20,000 sales: search and reports meet performance targets
- [ ] Urdu mode: no clipped Nastaliq text, numbers stay LTR
- [ ] License: valid code unlocks Pro; tampered code rejected; backup restore keeps data readable without Pro

Pilot acceptance (real shops)

- [ ] Use for 14 days in at least 3 mobile shops alongside the paper register
- [ ] End-of-day cash and customer balances match the register every day
- [ ] Zero unresolved accounting bugs before public release

Business validation (before fixing prices)

- [ ] Demo to at least 10 mobile-shop owners using their own records
- [ ] Count paid commitments (advance payment or signed pre-order), not just positive feedback
- [ ] Test Rs 1,499 vs Rs 1,999 for Mobile Shop Pro and measure setup-service demand
- [ ] Check whether the 3-tap sale target holds at a busy counter

11. Build plan with ready-to-paste prompts

Give the AI this whole document once, then send one milestone prompt at a time. Review and run the app on a real phone after each one before sending the next.

Order follows the phases in Section 3: 1 → 2 → 4 → 5 (Phase A), then 3 → 6 → 7 (Phase B), then 8 → 9 (Phase C), then 10 (Phase D). Prompt 1 sets up all three language files, but Urdu script screens are only tested in Phase C. Owner-PIN checks in Prompts 4 and 5 call a `PinGate` interface with a temporary always-allow stub, replaced by the real lock in Prompt 3. Never send the next prompt while any accounting or stock test is failing.

#	Milestone	Done when
1	Project foundation	App runs, theme, 3 languages, router, empty tabs

#	Milestone	Done when
2	Database and money core	All tables, migrations, Money type, posting engine, worked-example test passes
3	Onboarding, PIN, settings	First-launch flow, lock screen, demo data loader
4	Products and stock	Product CRUD, variants, barcodes, opening stock, adjustments, import
5	Sales (POS)	Full sale flow, receipts, returns, reversal
6	Purchases, parties, payments, expenses	Khata ledgers, allocations, reminders
7	Dashboard, Galla Band, reports	Home stats, cash closing, all core reports with PDF/CSV
8	Mobile Shop Pro	IMEI, used phones, repairs, qist
9	License, backup, security hardening	Activation, encrypted backup/restore, roles, health check
10	Polish and release	Performance, golden tests, icons, store assets, signed AAB

Prompt 1 — Foundation

```
Using the Smart Dukan spec (Sections 0, 4, 6 and 9), create a new Flutter project for Android only. Set up the folder structure exactly as in Section 6, Riverpod, go_router, Material 3 theme with all colour tokens and typography from Section 4 (bundle Inter, Noto Nastaliq Urdu, Noto Naskh Arabic), light/dark mode, and localization with English, Urdu (RTL) and Roman Urdu ARB files. Build the bottom navigation (Home, Sale, Khata, Stock, More) with the centre "+ Nayi Sale" button and placeholder screens. Add reusable widgets: StatCard, AmountKeypad, PartyTile, EmptyState, PrimaryButton, ConfirmSheet. Output run instructions and a screenshot description of each screen.
```

Prompt 2 — Database and posting engine

```
Implement Sections 7 and 8. Create all Drift tables with SQLCipher encryption, indexes, schema version 1 and a migration framework. Create the Money value type (int paisa). Build a PostingService that, inside one transaction, writes documents, stock_movements, stock_balances and a balanced journal entry for every event in the Section 8 table and every rule in Section 8.1 (opening balances, invoice extras, advances, sale and purchase returns, repairs, qist, cash closing), plus reversals. Write unit/integration tests for every row of that table and the full worked example, and the invariants list. Do not build UI yet.
```

Prompt 3 — Onboarding and security

```
Build Section 5.1 onboarding, PIN setup and lock screen with biometrics (Section 9), owner role creation, settings screen skeleton (Section 5.11), and a demo data loader for a mobile shop (30 products incl. IMEI phones, 10 customers, 3 suppliers, 2 weeks of sales) with a one-tap wipe.
```

Prompt 4 — Products and stock

```
Build Section 5.5: product list with filters, product form (variants, barcodes, units, IMEI flag, opening stock), product detail with movement history, stock adjustments with reasons and owner PIN, and Excel/CSV import with column mapping preview and error report. Use the PostingService for every stock change.
```

Prompt 5 — Sales

```
Build Section 5.3 New Sale exactly, including search by name/SKU/barcode/IMEI, camera scan, cart sheet, customer picker, Cash/Udhaar/Mixed/Wallet payment, double-tap protection, success screen, PDF receipt and WhatsApp share, sale return and reversal with owner PIN. Meet the target: known item cash sale in 3 taps.
```

Prompt 6 — Purchases, khata, payments, expenses

```
Build Sections 5.4, 5.6 and 5.7: purchases and returns, customer/supplier khata with running ledger, opening balances, credit limit warning, payments in/out with oldest-first allocation, WhatsApp reminder template from Section 9, statement PDF, and expenses with categories and photo.
```

Prompt 7 — Dashboard, cash closing, reports

```
Build Section 5.2 Home with live stats, Section 5.8 Galla Band with
denomination helper and over/short posting, and every report in Section 5.10
with date ranges, PDF and CSV export. Run heavy reports in an isolate and
meet Section 9 performance targets.
```

Prompt 8 — Mobile Shop Pro

```
Build Section 5.9: IMEI search and history, used phone purchase with private
CNIC/device photos and printable receipt, repair jobs with statuses, parts,
token slip and ready message, and qist plans with schedules, collection,
guarantor and overdue list on Home. Lock these behind the Mobile Shop Pro
license check but keep existing records readable.
```

Prompt 9 — License, backup, hardening

```
Implement Section 2 licensing: Device Code, Ed25519 verify with an embedded
public key, deep link smartdukan://activate, transfer screen, and
tools/license_gen.dart CLI that generates codes with a private key file.
Implement Section 9 encrypted backup/restore, staff roles and permissions,
owner-PIN gates, FLAG_SECURE, and a Health Check screen that runs all Section
8 invariants.
```

Prompt 10 — Polish and release

```
Run the full Section 10 checklist and fix every failure. Add golden tests,
app icon and splash (green brand), Play Store screenshots in Urdu and
English, privacy policy and license terms pages, release signing,
R8/shrinking, and build a release AAB. Report APK size, cold-start time and
any remaining issues.
```

Open decisions for the owner

- Final app name and Play Store availability check.
- Exact Pro price after pilot (test Rs 1,499 vs Rs 1,999).
- Whether to include Google Play billing at launch or only JazzCash/Easypaisa activation codes.